Import Libraries

In [1]:
```
import math
import os
import pprint
from collections import Counter
from contextlib import redirect_stdout

import matplotlib.animation as animation
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pylidc as pl
import seaborn as sns

from pylidc.utils import consensus
from skimage.measure import find_contours
from sqlalchemy import and_
from tabulate import tabulate
from tqdm import tqdm
```

```
/home/dityanugroho/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/
pylidc/__init__.py:27: UserWarning: pkg_resources is deprecated as an API. Se
https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources pa
is slated for removal as early as 2025-11-30. Refrain from using this package
pin to Setuptools<81.
  import pkg_resources as _pr
```

In [2]:
```
sns.set_theme(
    style="whitegrid",
    palette="Set2",
    context="notebook"
)
```

Explore NBIA Digest Data (XLSX)

Load Data

In [3]:
```
nbia_digest_df = pd.read_excel("../../000_dataset/_lidc/000_metadata/000_
nbia_digest_df.columns = (
    nbia_digest_df.columns
    .str.lower()
    .str.replace(" ", "_", regex=False)
)

print(f"{'='*20} DataFrame Information {'='*20}")
nbia_digest_df.info()
print(f"\n{'='*20} First 5 Rows Of A DataFrame {'='*20}")
display(nbia_digest_df.head())
```

```
==================== DataFrame Information ====================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1308 entries, 0 to 1307
Data columns (total 36 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   patient_id                              1308 non-null   object
 1   patient_name                            0 non-null      float64
 2   patient_birth_date                      0 non-null      float64
 3   patient_sex                             451 non-null    object
 4   ethnic_group                            61 non-null     object
 5   phantom                                 1308 non-null   object
 6   species_code                            1308 non-null   int64
 7   species_description                     1308 non-null   object
 8   study_instance_uid                      1308 non-null   object
 9   study_date                              1308 non-null   object
 10  study_description                       617 non-null    object
 11  admitting_diagnosis_description         0 non-null      float64
 12  study_id                                10 non-null     float64
 13  patient_age                             335 non-null    object
 14  longitudinal_temporal_event_type        0 non-null      float64
 15  longitudinal_temporal_offset_from_event  0 non-null      float64
 16  series_instance_uid                     1308 non-null   object
 17  project                                 1308 non-null   object
 18  modality                                1308 non-null   object
 19  protocol_name                           204 non-null    object
 20  series_date                             1308 non-null   object
 21  series_description                      464 non-null    object
 22  body_part_examined                      1308 non-null   object
 23  series_number                           1174 non-null   float64
 24  annotations_flag                        1161 non-null   float64
 25  manufacturer                            1308 non-null   object
 26  manufacturer_model_name                 1308 non-null   object
 27  software_versions                       1224 non-null   object
 28  image_count                             1308 non-null   int64
 29  max_submission_timestamp                1308 non-null   object
 30  license_name                            1308 non-null   object
 31  license_uri                             1308 non-null   object
 32  collection_uri                          1308 non-null   object
 33  file_size                               1308 non-null   int64
 34  date_released                           1308 non-null   object
 35  third_party_analysis                    0 non-null      float64
dtypes: float64(9), int64(3), object(24)
memory usage: 368.0+ KB

==================== First 5 Rows Of A DataFrame ====================
```

	patient_id	patient_name	patient_birth_date	patient_sex	ethnic_group	phantom	specie
0	LIDC-IDRI-0001	NaN	NaN	NaN	NaN	NO	337
1	LIDC-IDRI-0001	NaN	NaN	NaN	NaN	NO	337
2	LIDC-IDRI-0002	NaN	NaN	NaN	NaN	NO	337
3	LIDC-IDRI-0002	NaN	NaN	NaN	NaN	NO	337
4	LIDC-IDRI-0003	NaN	NaN	NaN	NaN	NO	337

5 rows × 36 columns

EDA Dashboard

In [4]:
```
# Filter CT only
ct_df = nbia_digest_df[nbia_digest_df["modality"] == "CT"].copy()

# Dashboard Layout
n_charts = 6
n_cols = 3
n_rows = math.ceil(n_charts / n_cols)

fig, axes = plt.subplots(
    n_rows,
    n_cols,
    figsize=(6 * n_cols, 5 * n_rows)
)

fig.suptitle(
    "Exploratory Data Analysis of LIDC-IDRI NBIA Digest",
    fontsize=16,
    fontweight="bold"
)

axes = axes.flatten()

# Imaging Modalities
ax = axes[0]

sns.countplot(
    data=nbia_digest_df,
    x="modality",
```

```
    hue="modality",
    order=nbia_digest_df["modality"].value_counts().index,
    legend=False,
    ax=ax
)

ax.set_title("Distribution of Imaging Modalities")
ax.set_xlabel("Modality")
ax.set_ylabel("Number of Series")

for container in ax.containers:
    ax.bar_label(container)

sns.despine(ax=ax)

# CT Manufacturers
ax = axes[4]

manufacturer_counts = (
    ct_df["manufacturer"]
    .value_counts()
    .sort_values()
)

sns.barplot(
    x=manufacturer_counts.values,
    y=manufacturer_counts.index,
    ax=ax
)

ax.set_title("Distribution of CT Manufacturers")
ax.set_xlabel("Number of CT Series")
ax.set_ylabel("Manufacturer")

for container in ax.containers:
    ax.bar_label(container, padding=3)

sns.despine(ax=ax)

# Top 10 CT Scanner Models
ax = axes[2]

scanner_model_counts = (
    ct_df["manufacturer_model_name"]
    .value_counts()
    .head(10)
    .sort_values()
)

sns.barplot(
    x=scanner_model_counts.values,
    y=scanner_model_counts.index,
    ax=ax
)

ax.set_title("Top 10 CT Scanner Models")
ax.set_xlabel("Number of CT Series")
ax.set_ylabel("Scanner Model")

for container in ax.containers:
```

```
    ax.bar_label(container, padding=3)

sns.despine(ax=ax)

# Top 10 CT Software Versions
ax = axes[3]

software_counts = (
    ct_df["software_versions"]
    .value_counts()
    .head(10)
    .sort_values()
)

sns.barplot(
    x=software_counts.values,
    y=software_counts.index,
    ax=ax
)

ax.set_title("Top 10 CT Software Versions")
ax.set_xlabel("Number of CT Series")
ax.set_ylabel("Software Version")

for container in ax.containers:
    ax.bar_label(container, padding=3)

sns.despine(ax=ax)

# Distribution of CT Image Count
ax = axes[1]

sns.histplot(
    data=ct_df,
    x="image_count",
    bins=30,
    kde=True,
    ax=ax
)

ax.set_title("Distribution of CT Image Count")
ax.set_xlabel("Number of Images")
ax.set_ylabel("Frequency")

sns.despine(ax=ax)

# Top 10 CT Scanner Configurations
ax = axes[5]

ct_df["scanner_configuration"] = (
    ct_df["manufacturer"] +
    " - " +
    ct_df["manufacturer_model_name"]
)

scanner_config_counts = (
    ct_df["scanner_configuration"]
    .value_counts()
    .head(10)
    .sort_values()
```

```
)

sns.barplot(
    x=scanner_config_counts.values,
    y=scanner_config_counts.index,
    ax=ax
)

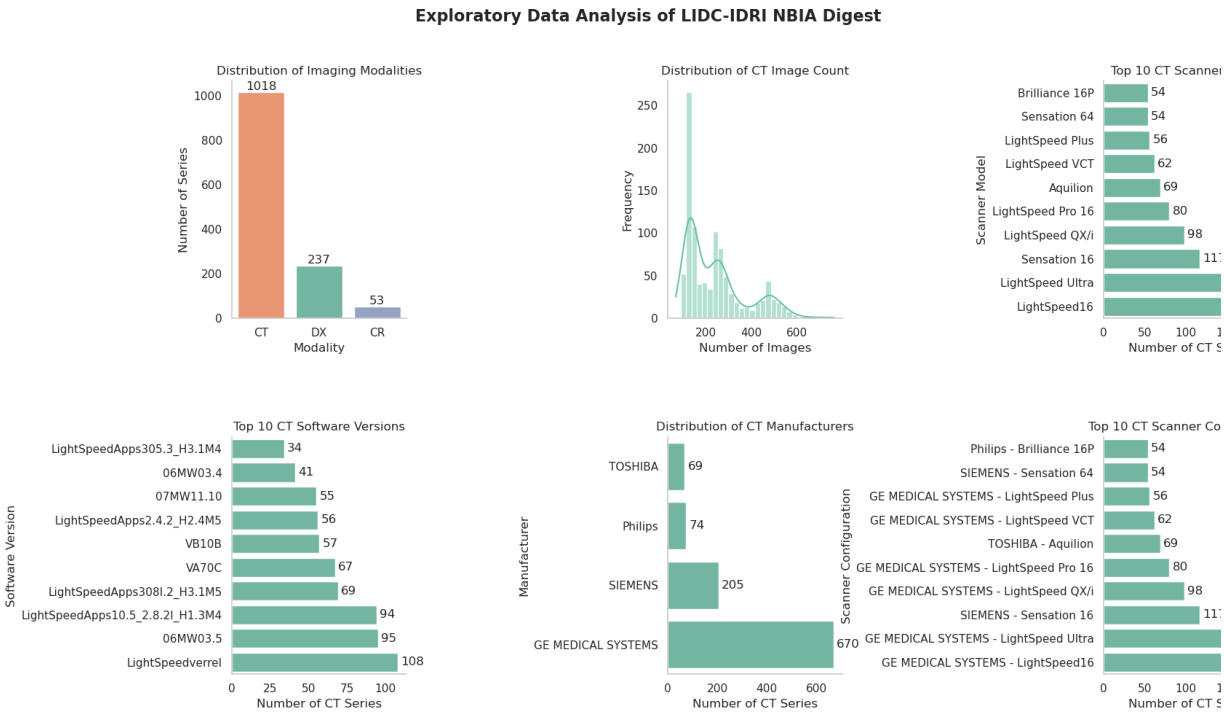
ax.set_title("Top 10 CT Scanner Configurations")
ax.set_xlabel("Number of CT Series")
ax.set_ylabel("Scanner Configuration")

for container in ax.containers:
    ax.bar_label(container, padding=3)

sns.despine(ax=ax)

# Final Layout
for ax in axes:
    ax.grid(False)

fig.tight_layout(rect=[0, 0, 1, 0.96])
fig.subplots_adjust(
    wspace=1.5,
    hspace=0.5
)
plt.show()
```



- CT is the dominant imaging modality in the LIDC-IDRI collection, accounting for 1,018 series DX (237) and CR (53) comprise a much smaller portion of the dataset.
- The distribution of CT image counts shows noticeable variability, with most CT series contai approximately 100–300 slices, while a smaller number of scans contain more than 500 slice

Explore Nodule Counts Data (XLSX)

Load Data

- *total number of lesions that received either a "nodule < 3mm" mark or a "nodule >= 3mm" r from at least one of the four LIDC radiologists
- **total number of lesions that received a "nodule >= 3mm" mark from at least one of the four radiologists (regardless of how the other radiologists marked the lesion)
- ***total number of lesions that received a "nodule < 3mm" mark from at least one of the four radiologists (with no radiologist assigning the lesion a "nodule >= 3mm" mark)

In [5]:
```
nodule_counts = pd.read_excel("../../000_dataset/_lidc/000_metadata/000_l
nodule_counts.columns = (
    nodule_counts.columns
    .str.lower()
    .str.replace(" ", "_", regex=False)
)

print(f"{'='*20} DataFrame Information {'='*20}")
nodule_counts.info()
print(f"\n{'='*20} First 5 Rows Of A DataFrame {'='*20}")
display(nodule_counts.head())
```

```
==================== DataFrame Information ====================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1018 entries, 0 to 1017
Data columns (total 4 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   tcia_patent_id             1018 non-null   object
 1   total_number_of_nodules*   1018 non-null   int64
 2   number_of_nodules_>=3mm**  1018 non-null   int64
 3   number_of_nodules_<3mm***  1018 non-null   int64
dtypes: int64(3), object(1)
memory usage: 31.9+ KB

==================== First 5 Rows Of A DataFrame ====================
```

	tcia_patent_id	total_number_of_nodules*	number_of_nodules_>=3mm**	number_of_nodu
0	LIDC-IDRI-0001	4	1	
1	LIDC-IDRI-0002	12	1	
2	LIDC-IDRI-0003	4	4	
3	LIDC-IDRI-0004	4	1	
4	LIDC-IDRI-0005	9	3	

EDA Dashboard

In [6]:
```
# Dashboard Layout
n_charts = 4
```

```
n_cols = 2
n_rows = math.ceil(n_charts / n_cols)

fig, axes = plt.subplots(
    n_rows,
    n_cols,
    figsize=(14, 10)
)

fig.suptitle(
    "Exploratory Data Analysis of LIDC-IDRI Nodule Counts",
    fontsize=16,
    fontweight="bold"
)

axes = axes.flatten()

# Distribution of Total Number of Nodules
ax = axes[1]

sns.histplot(
    data=nodule_counts,
    x="total_number_of_nodules*",
    bins=20,
    kde=True,
    color=sns.color_palette("Set2")[0],
    ax=ax
)

ax.set_title("Distribution of Total Number of Nodules")
ax.set_xlabel("Number of Nodules")
ax.set_ylabel("Number of Patients")

sns.despine(ax=ax)

# Total Nodules by Size Category
ax = axes[0]

large_nodules = nodule_counts["number_of_nodules_>=3mm**"].sum()
small_nodules = nodule_counts["number_of_nodules_<3mm***"].sum()

labels = ["Nodules ≥ 3 mm", "Nodules < 3 mm"]
counts = [large_nodules, small_nodules]

total_nodules = sum(counts)

bars = ax.bar(
    labels,
    counts,
    color=sns.color_palette("Set2")[:2]
)

ax.set_title("Total Nodules by Size Category")
ax.set_xlabel("Nodule Category")
ax.set_ylabel("Total Number of Nodules")

# Add count and percentage
for bar, value in zip(bars, counts):
    percentage = value / total_nodules * 100
```

```
    ax.text(
        bar.get_x() + bar.get_width()/2,
        value,
        f"{value:,}\n({percentage:.1f}%)",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold"
    )

sns.despine(ax=ax)

# Distribution of Nodules ≥ 3 mm
ax = axes[2]

sns.histplot(
    data=nodule_counts,
    x="number_of_nodules_>=3mm**",
    bins=15,
    kde=True,
    color=sns.color_palette("Set2")[2],
    ax=ax
)

ax.set_title("Distribution of Nodules ≥ 3 mm")
ax.set_xlabel("Number of Nodules")
ax.set_ylabel("Number of Patients")

sns.despine(ax=ax)

# Distribution of Nodules < 3 mm
ax = axes[3]

sns.histplot(
    data=nodule_counts,
    x="number_of_nodules_<3mm***",
    bins=15,
    kde=True,
    color=sns.color_palette("Set2")[3],
    ax=ax
)

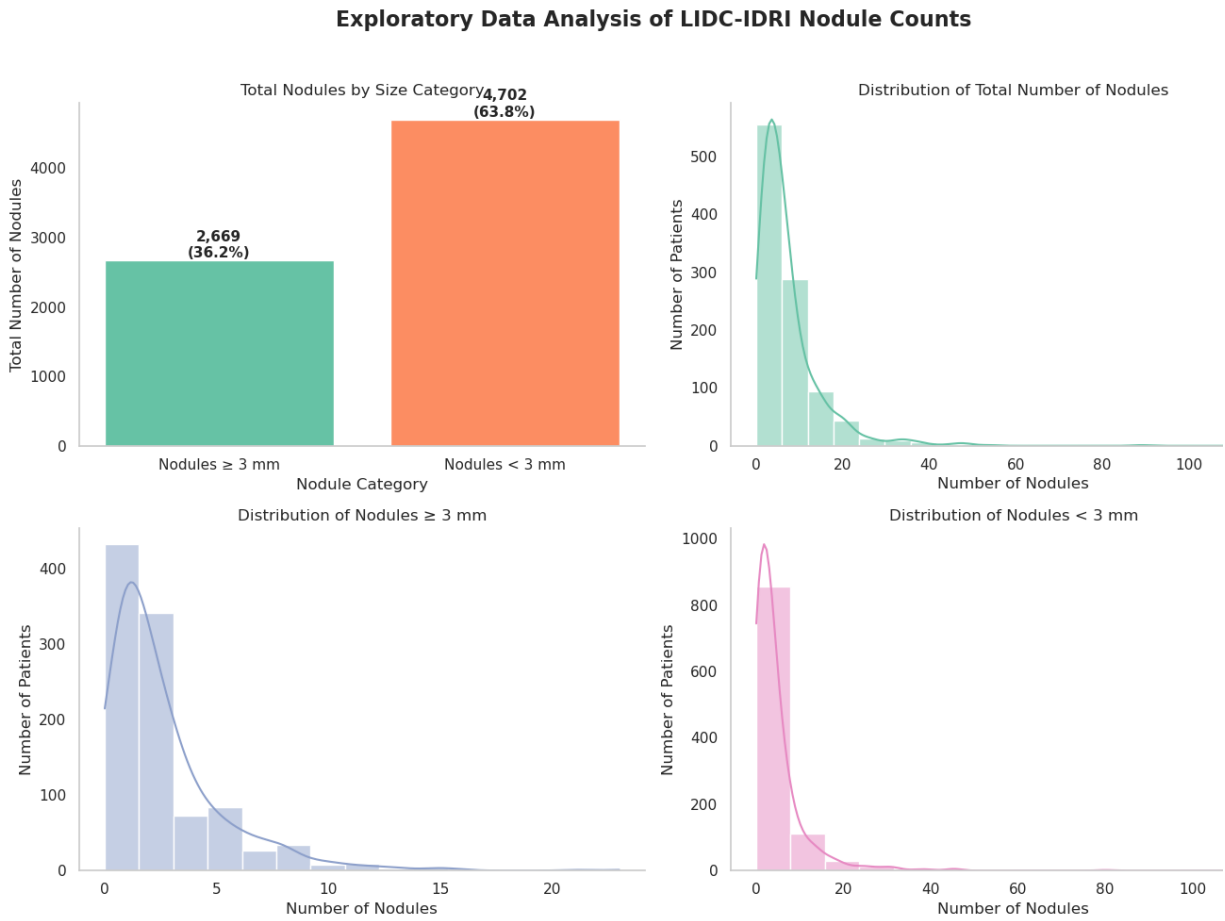
ax.set_title("Distribution of Nodules < 3 mm")
ax.set_xlabel("Number of Nodules")
ax.set_ylabel("Number of Patients")

sns.despine(ax=ax)

# Final Layout
for ax in axes:
    ax.grid(False)

fig.tight_layout(rect=[0, 0, 1, 0.96])

plt.show()
```



- The original LIDC-IDRI dataset contains more nodules smaller than 3 mm (63.8%) than nod measuring at least 3 mm (36.2%).
- The total number of nodules per patient is highly right-skewed, with most patients having onl nodules.
- Most patients contain only one to three nodules measuring at least 3 mm, while patients witl large nodules are relatively uncommon.
- Most patients contain only a few nodules smaller than 3 mm, although a small number of pa have substantially more such nodules.

Explore LIDC-IDRI Dataset

Load Dataset

In [7]:
```
# Load All Scans
scans = pl.query(pl.Scan).all()
print(f"Total scan objects loaded: {len(scans)}")
```

```
Total scan objects loaded: 1018
```

Total Studies and Unique Patient from LIDC-IDRI Datas

In [8]:
```
# Extract patient ids
patient_ids = [scan.patient_id for scan in scans]
```

```
# Count total studies
total_studies = len(patient_ids)

# Get unique patient ids
unique_patients = sorted(set(patient_ids))
total_unique_patients = len(unique_patients)

# Count occurrences of each patient id
scan_count_per_patient = Counter(patient_ids)

# Create summary dataframe
summary_df = pd.DataFrame({
    "Metric": [
        "Total Studies",
        "Total Unique Patients"
    ],
    "Value": [
        total_studies,
        total_unique_patients
    ]
})

display(summary_df)
```

	Metric	Value
0	Total Studies	1018
1	Total Unique Patients	1010

Check Patient with Multiple Studies

In [9]:

```
# Find duplicated patient ids automatically
duplicate_patients = {
    pid: count
    for pid, count in scan_count_per_patient.items()
    if count > 1
}

# Automatically get duplicated patient ids
duplicate_ids = list(duplicate_patients.keys())
study_uids = []

for pid in duplicate_ids:
    scans_pid = pl.query(pl.Scan).filter(
        pl.Scan.patient_id == pid
    ).all()

    for i, scan in enumerate(scans_pid):
        study_uids.append(scan.study_instance_uid)

# Duplicate patient ids data
duplicate_ids_ct_df = nbia_digest_df[
    nbia_digest_df["study_instance_uid"].isin(study_uids)
]
duplicate_ids_ct_df = duplicate_ids_ct_df[["patient_id", "study_instance_
display(duplicate_ids_ct_df)
```

	patient_id	study_instance_uid	modality	series_description
234	LIDC-IDRI-0132	1.3.6.1.4.1.14519.5.2.1.6279.6001.314138616411...	CT	ThorRoutine 3.0 B31f
235	LIDC-IDRI-0132	1.3.6.1.4.1.14519.5.2.1.6279.6001.218658642102...	CT	ChestRoutine 3.0 B31f
263	LIDC-IDRI-0151	1.3.6.1.4.1.14519.5.2.1.6279.6001.780121538270...	CT	NaN
264	LIDC-IDRI-0151	1.3.6.1.4.1.14519.5.2.1.6279.6001.222444895275...	CT	ThoraxRoutine 3.0 B31s
556	LIDC-IDRI-0315	1.3.6.1.4.1.14519.5.2.1.6279.6001.190722769298...	CT	ThorRoutine 3.0 B31f
557	LIDC-IDRI-0315	1.3.6.1.4.1.14519.5.2.1.6279.6001.709924281509...	CT	NaN
572	LIDC-IDRI-0332	1.3.6.1.4.1.14519.5.2.1.6279.6001.124508094927...	CT	NaN
573	LIDC-IDRI-0332	1.3.6.1.4.1.14519.5.2.1.6279.6001.299122687641...	CT	NaN
594	LIDC-IDRI-0355	1.3.6.1.4.1.14519.5.2.1.6279.6001.221138977753...	CT	NaN
595	LIDC-IDRI-0355	1.3.6.1.4.1.14519.5.2.1.6279.6001.246541809203...	CT	ThoraxRoutine 3.0 B31s
604	LIDC-IDRI-0365	1.3.6.1.4.1.14519.5.2.1.6279.6001.212341120080...	CT	ThoraxRoutine 3.0 B31f
605	LIDC-IDRI-0365	1.3.6.1.4.1.14519.5.2.1.6279.6001.216207548522...	CT	NaN
674	LIDC-IDRI-0442	1.3.6.1.4.1.14519.5.2.1.6279.6001.193438875802...	CT	ThoraxRoutine 3.0 B31s
675	LIDC-IDRI-0442	1.3.6.1.4.1.14519.5.2.1.6279.6001.265657827498...	CT	ThorRoutine 3.0 B31f
711	LIDC-IDRI-0484	1.3.6.1.4.1.14519.5.2.1.6279.6001.202012743926...	CT	ChestRoutine 3.0 B31f
712	LIDC-IDRI-0484	1.3.6.1.4.1.14519.5.2.1.6279.6001.292226263060...	CT	ChestRoutine 3.0 B31f

Duplicate Patient Study Analysis

In [10]:
```python
# Store metadata for patients with multiple CT scans
duplicate_scan_info = []

# Iterate through duplicated patient IDs
for pid in duplicate_ids:
    scans = pl.query(pl.Scan).filter(
        pl.Scan.patient_id == pid
    ).all()

    # Extract metadata from each CT scan
    for scan in scans:
        duplicate_scan_info.append({
            "patient_id": scan.patient_id,
            "study_uid": scan.study_instance_uid,
            "series_uid": scan.series_instance_uid,
            "slice_thickness": scan.slice_thickness,
```

```
            "pixel_spacing": scan.pixel_spacing,
        })

# Convert the collected metadata into a DataFrame
duplicate_scan_info_df = pd.DataFrame(duplicate_scan_info)

# Display duplicate scan metadata
display(duplicate_scan_info_df)
```

	patient_id	study_uid	
0	LIDC-IDRI-0132	1.3.6.1.4.1.14519.5.2.1.6279.6001.218658642102...	1.3.6.1.4.1.14519.5.2.1.6279.600
1	LIDC-IDRI-0132	1.3.6.1.4.1.14519.5.2.1.6279.6001.314138616411...	1.3.6.1.4.1.14519.5.2.1.6279.600
2	LIDC-IDRI-0151	1.3.6.1.4.1.14519.5.2.1.6279.6001.222444895275...	1.3.6.1.4.1.14519.5.2.1.6279.600
3	LIDC-IDRI-0151	1.3.6.1.4.1.14519.5.2.1.6279.6001.780121538270...	1.3.6.1.4.1.14519.5.2.1.6279.600
4	LIDC-IDRI-0315	1.3.6.1.4.1.14519.5.2.1.6279.6001.190722769298...	1.3.6.1.4.1.14519.5.2.1.6279.600
5	LIDC-IDRI-0315	1.3.6.1.4.1.14519.5.2.1.6279.6001.709924281509...	1.3.6.1.4.1.14519.5.2.1.6279.600
6	LIDC-IDRI-0332	1.3.6.1.4.1.14519.5.2.1.6279.6001.124508094927...	1.3.6.1.4.1.14519.5.2.1.6279.600
7	LIDC-IDRI-0332	1.3.6.1.4.1.14519.5.2.1.6279.6001.299122687641...	1.3.6.1.4.1.14519.5.2.1.6279.600
8	LIDC-IDRI-0355	1.3.6.1.4.1.14519.5.2.1.6279.6001.221138977753...	1.3.6.1.4.1.14519.5.2.1.6279.600
9	LIDC-IDRI-0355	1.3.6.1.4.1.14519.5.2.1.6279.6001.246541809203...	1.3.6.1.4.1.14519.5.2.1.6279.600
10	LIDC-IDRI-0365	1.3.6.1.4.1.14519.5.2.1.6279.6001.212341120080...	1.3.6.1.4.1.14519.5.2.1.6279.600
11	LIDC-IDRI-0365	1.3.6.1.4.1.14519.5.2.1.6279.6001.216207548522...	1.3.6.1.4.1.14519.5.2.1.6279.600
12	LIDC-IDRI-0442	1.3.6.1.4.1.14519.5.2.1.6279.6001.193438875802...	1.3.6.1.4.1.14519.5.2.1.6279.600
13	LIDC-IDRI-0442	1.3.6.1.4.1.14519.5.2.1.6279.6001.265657827498...	1.3.6.1.4.1.14519.5.2.1.6279.600
14	LIDC-IDRI-0484	1.3.6.1.4.1.14519.5.2.1.6279.6001.202012743926...	1.3.6.1.4.1.14519.5.2.1.6279.600
15	LIDC-IDRI-0484	1.3.6.1.4.1.14519.5.2.1.6279.6001.292226263060...	1.3.6.1.4.1.14519.5.2.1.6279.600

In [11]:
```
# Summarize duplicate CT study information for each patient
summary = []

# Iterate through duplicated patient IDs
for pid in duplicate_ids:

    # Retrieve metadata for the current patient
    df = duplicate_scan_info_df[
        duplicate_scan_info_df["patient_id"] == pid
    ]
```

```
    # Count unique studies, series, and slice thickness values
    summary.append({
        "patient_id": pid,
        "n_studies": df["study_uid"].nunique(),
        "n_series": df["series_uid"].nunique(),
        "slice_thickness": df["slice_thickness"].nunique(),
    })

# Convert the summary into a DataFrame
summary_df = pd.DataFrame(summary)

# Display the summary
display(summary_df)
```

	patient_id	n_studies	n_series	slice_thickness
0	LIDC-IDRI-0132	2	2	1
1	LIDC-IDRI-0151	2	2	1
2	LIDC-IDRI-0315	2	2	1
3	LIDC-IDRI-0332	2	2	1
4	LIDC-IDRI-0355	2	2	1
5	LIDC-IDRI-0365	2	2	1
6	LIDC-IDRI-0442	2	2	1
7	LIDC-IDRI-0484	2	2	1

In [12]:
```
# Store annotation information for duplicate CT studies
results = []

# Iterate through duplicated patient IDs
for pid in duplicate_ids:
    scans = pl.query(pl.Scan).filter(
        pl.Scan.patient_id == pid
    ).all()

    # Count the number of annotations and annotation clusters for each CT
    for scan in scans:
        results.append({
            "patient_id": scan.patient_id,
            "study_uid": scan.study_instance_uid,
            "annotations": len(scan.annotations),
            "clusters": len(scan.cluster_annotations())
        })

# Convert the collected information into a DataFrame
results_df = pd.DataFrame(results)

# Display the annotation summary
display(results_df)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[12], line 16
     10     # Count the number of annotations and annotation clusters for eac
study
     11     for scan in scans:
     12         results.append({
     13             "patient_id": scan.patient_id,
     14             "study_uid": scan.study_instance_uid,
     15             "annotations": len(scan.annotations),
---> 16             "clusters": len(scan.cluster_annotations())
     17         })
     19 # Convert the collected information into a DataFrame
     20 results_df = pd.DataFrame(results)

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/pylidc/
Scan.py:439, in Scan.cluster_annotations(self, metric, tol, factor, min_tol,
return_distance_matrix, verbose)
    437 for i in range(N):
    438     for j in range(i+1,N):
--> 439         D[i,j] = D[j,i] = metric(self.annotations[i],
    440                                  self.annotations[j])
    442 adjacency = D <= tol
    443 nnods, cids = connected_components(adjacency, directed=False)

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/pylidc/
annotation_distance_metrics.py:27, in <lambda>(a, b)
     24     else:
     25         raise ValueError('invalid `which` value.')
---> 27 metrics['min'] = lambda a,b: pairdist(a,b,'min')
     28 metrics['max'] = lambda a,b: pairdist(a,b,'max')
     29 metrics['avg'] = lambda a,b: pairdist(a,b,'avg')

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/pylidc/
annotation_distance_metrics.py:15, in pairdist(ann1, ann2, which)
      6 def pairdist(ann1, ann2, which):
      7     """
      8     Compute the pairwise euclidean distance between
      9     the contour boundary points, and return the
   (...)
     13         One of 'min', 'max', or 'avg'.
     14     """
---> 15     dists = cdist(ann1.contours_matrix,
     16                   ann2.contours_matrix)
     18     if   which == 'min':
     19         return dists.min()

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/pylidc/
Annotation.py:968, in Annotation.contours_matrix(self)
    963 @property
    964 def contours_matrix(self):
    965     """
    966     All the contour index values a 3D numpy array.
    967     """
--> 968     return np.vstack([c.to_matrix(include_k=True)
    969                       for c in sorted(self.contours,
    970                                       key=lambda c: c.image_z_position)

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/pylidc/
Annotation.py:968, in <listcomp>(.0)
```

```
    963 @property
    964 def contours_matrix(self):
    965     """
    966     All the contour index values a 3D numpy array.
    967     """
--> 968     return np.vstack([c.to_matrix(include_k=True)
    969                       for c in sorted(self.contours,
    970                                       key=lambda c: c.image_z_position)

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/pylidc/
Contour.py:111, in Contour.to_matrix(self, include_k)
    109 k  = np.ones(ij.shape[0])*self.image_k_position
    110 zs = self.annotation.contour_slice_zvals
--> 111 return np.c_[ij, k].astype(np.int)

File ~/miniforge3/envs/nodule_py310/lib/python3.10/site-packages/numpy/
__init__.py:397, in __getattr__(attr)
    392     warnings.warn(
    393         f"In the future `np.{attr}` will be defined as the "
    394         "corresponding NumPy scalar.", FutureWarning, stacklevel=2)
    396 if attr in __former_attrs__:
--> 397     raise AttributeError(__former_attrs__[attr], name=None)
    399 if attr in __expired_attributes__:
    400     raise AttributeError(
    401         f"`np.{attr}` was removed in the NumPy 2.0 release. "
    402         f"{__expired_attributes__[attr]}",
    403         name=None
    404     )

AttributeError: module 'numpy' has no attribute 'int'.
`np.int` was a deprecated alias for the builtin `int`. To avoid this error in
existing code, use `int` by itself. Doing this will not modify any behavior a
safe. When replacing `np.int`, you may wish to use e.g. `np.int64` or `np.int
specify the precision. If you wish to review your current use, check the rele
note link for additional information.
The aliases was originally deprecated in NumPy 1.20; for more details and gui
see the original release note at:
    https://numpy.org/devdocs/release/1.20.0-notes.html#deprecations
```

Eight patients in the LIDC-IDRI dataset have two CT studies with different Study Instance UIDs a Series Instance UIDs. The slice thickness is identical within each patient. One CT study should b selected for each patient to ensure a unique patient representation in the final dataset.

Study Selection Criteria

- Priority was given to the study with the largest number of nodule annotation clusters (len(scan.cluster_annotations())), as annotation clusters represent unique pulmonary nodule
- If multiple studies contained the same number of annotation clusters, the study with the larg number of individual annotations (len(scan.annotations)) was selected.
- If both the number of annotation clusters and individual annotations were identical, the study largest number of image slices (image_count) was selected to retain the more complete CT

In [ ]:
```
excluded_study_uids = [
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.218658642102832118810712329678",
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.222444895275634974977432374937",
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.709924281509506265799179464374",
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.124508094927772723365648607398",
```

```
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.221138977753587043062153841454",
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.216207548522622026268886920069",
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.265657827498396379702226874113",
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.202012743926377385386645171009",
]
```

LIDC-IDRI Studies Metadata Exploration

DICOM Metadata Extraction

In [ ]:

```
# # Helper Functions
# def get_dicom_attr(dicom, attr, default=np.nan):
#     """Safely retrieve a DICOM attribute."""
#     return getattr(dicom, attr, default)


# # Extract DICOM Metadata
# study_metadata_list = []
# failed_studies = []

# scans = pl.query(pl.Scan).all()

# for scan in tqdm(scans, desc="Processing CT studies"):
#     try:
#         dicom_images = scan.load_all_dicom_images()

#         # Basic information
#         first_dicom = dicom_images[0]
#         middle_dicom = dicom_images[len(dicom_images) // 2]

#         # Slice positions
#         z_positions = sorted(
#             float(dcm.ImagePositionPatient[2])
#             for dcm in dicom_images
#         )

#         slice_spacing = (
#             float(np.mean(np.diff(z_positions)))
#             if len(z_positions) > 1
#             else np.nan
#         )

#         scan_length = (
#             float(max(z_positions) - min(z_positions))
#             if len(z_positions) > 1
#             else np.nan
#         )

#         # Convert CT volume to Hounsfield Unit (HU)
#         volume = scan.to_volume()

#         hu_min = float(np.min(volume))
#         hu_max = float(np.max(volume))
#         hu_mean = float(np.mean(volume))
#         hu_std = float(np.std(volume))
```

```
#         hu_p1 = float(np.percentile(volume, 1))
#         hu_p5 = float(np.percentile(volume, 5))
#         hu_p95 = float(np.percentile(volume, 95))
#         hu_p99 = float(np.percentile(volume, 99))

#         # Store metadata
#         metadata = {

#             # Identification
#             "patient_id": scan.patient_id,
#             "study_instance_uid": scan.study_instance_uid,
#             "series_instance_uid": scan.series_instance_uid,

#             # Image Geometry
#             "rows": int(first_dicom.Rows),
#             "columns": int(first_dicom.Columns),
#             "num_slices": len(dicom_images),

#             "pixel_spacing_x_mm":
#                 float(first_dicom.PixelSpacing[0]),

#             "pixel_spacing_y_mm":
#                 float(first_dicom.PixelSpacing[1]),

#             "slice_thickness_mm":
#                 float(get_dicom_attr(
#                     middle_dicom,
#                     "SliceThickness"
#                 )),

#             "slice_spacing_mm":
#                 slice_spacing,

#             "scan_length_mm":
#                 scan_length,

#             # Intensity
#             "rescale_slope":
#                 float(get_dicom_attr(
#                     middle_dicom,
#                     "RescaleSlope",
#                     1.0
#                 )),

#             "rescale_intercept":
#                 float(get_dicom_attr(
#                     middle_dicom,
#                     "RescaleIntercept",
#                     0.0
#                 )),

#             # HU Statistics
#             "hu_min": hu_min,
#             "hu_max": hu_max,
#             "hu_mean": hu_mean,
#             "hu_std": hu_std,
#             "hu_p1": hu_p1,
#             "hu_p5": hu_p5,
#             "hu_p95": hu_p95,
#             "hu_p99": hu_p99,
```

```
#             # Reconstruction
#             "convolution_kernel":
#                 get_dicom_attr(
#                     middle_dicom,
#                     "ConvolutionKernel"
#                 ),

#             "reconstruction_diameter_mm":
#                 get_dicom_attr(
#                     middle_dicom,
#                     "ReconstructionDiameter"
#                 ),

#             # Patient Orientation
#             "patient_position":
#                 get_dicom_attr(
#                     middle_dicom,
#                     "PatientPosition"
#                 ),
#         }

#         study_metadata_list.append(metadata)

#     except Exception as e:

#         failed_studies.append({
#             "patient_id": scan.patient_id,
#             "study_instance_uid": scan.study_instance_uid,
#             "error": str(e)
#         })


# # Convert to DataFrame
# study_metadata_df = pd.DataFrame(study_metadata_list)

# # Save CSV
# output_file = (
#     "../../000_dataset/_lidc/000_metadata/000_dicom_metadata.csv"
# )

# study_metadata_df.to_csv(
#     output_file,
#     index=False
# )

# # Summary
# print(f"Metadata saved successfully to: {output_file}")
# print(f"Processed studies : {len(study_metadata_df)}")
# print(f"Failed studies    : {len(failed_studies)}")

# if failed_studies:
#     failed_df = pd.DataFrame(failed_studies)
#     display(failed_df.head())
```

Load DICOM Metadata

In [ ]:

```python
dicom_metadata_df = pd.read_csv("../../000_dataset/_lidc/000_metadata/000

print(f"{'='*20} DataFrame Information {'='*20}")
dicom_metadata_df.info()
print(f"\n{'='*20} First 5 Rows Of A DataFrame {'='*20}")
display(dicom_metadata_df.head())
```

```
==================== DataFrame Information ====================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1018 entries, 0 to 1017
Data columns (total 24 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   patient_id                   1018 non-null   object
 1   study_instance_uid           1018 non-null   object
 2   series_instance_uid          1018 non-null   object
 3   rows                         1018 non-null   int64
 4   columns                      1018 non-null   int64
 5   num_slices                   1018 non-null   int64
 6   pixel_spacing_x_mm           1018 non-null   float64
 7   pixel_spacing_y_mm           1018 non-null   float64
 8   slice_thickness_mm           1018 non-null   float64
 9   slice_spacing_mm             1018 non-null   float64
 10  scan_length_mm               1018 non-null   float64
 11  rescale_slope                1018 non-null   float64
 12  rescale_intercept            1018 non-null   float64
 13  hu_min                       1018 non-null   float64
 14  hu_max                       1018 non-null   float64
 15  hu_mean                      1018 non-null   float64
 16  hu_std                       1018 non-null   float64
 17  hu_p1                        1018 non-null   float64
 18  hu_p5                        1018 non-null   float64
 19  hu_p95                       1018 non-null   float64
 20  hu_p99                       1018 non-null   float64
 21  convolution_kernel           1018 non-null   object
 22  reconstruction_diameter_mm   1018 non-null   float64
 23  patient_position             1018 non-null   object
dtypes: float64(16), int64(3), object(5)
memory usage: 191.0+ KB

==================== First 5 Rows Of A DataFrame ====================
```

	patient_id	study_instance_uid	seri
0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.2.1.6279.6001
1	LIDC-IDRI-0069	1.3.6.1.4.1.14519.5.2.1.6279.6001.303241414168...	1.3.6.1.4.1.14519.5.2.1.6279.6001
2	LIDC-IDRI-0079	1.3.6.1.4.1.14519.5.2.1.6279.6001.303921580531...	1.3.6.1.4.1.14519.5.2.1.6279.6001
3	LIDC-IDRI-0101	1.3.6.1.4.1.14519.5.2.1.6279.6001.584233139051...	1.3.6.1.4.1.14519.5.2.1.6279.6001
4	LIDC-IDRI-0110	1.3.6.1.4.1.14519.5.2.1.6279.6001.924939006160...	1.3.6.1.4.1.14519.5.2.1.6279.6001

5 rows × 24 columns

Descriptive Statistics

In [ ]:
```
dicom_metadata_df.describe()
```

Out[ ]:

	rows	columns	num_slices	pixel_spacing_x_mm	pixel_spacing_y_mm	slice_th
count	1018.0	1018.0	1018.000000	1018.000000	1018.000000	
mean	512.0	512.0	239.631631	0.687689	0.687689	
std	0.0	0.0	133.457158	0.084851	0.084851	
min	512.0	512.0	65.000000	0.460938	0.460938	
25%	512.0	512.0	133.000000	0.625000	0.625000	
50%	512.0	512.0	203.500000	0.698609	0.698609	
75%	512.0	512.0	289.750000	0.742188	0.742188	
max	512.0	512.0	764.000000	0.976562	0.976562	

- Image Size Consistency:

  All CT studies have a uniform image size of 512 × 512 pixels, no image size standardization required. Image resizing is only necessary if the segmentation or classification model requir smaller input.

- Number of Slices:

  The number of slices varies considerably across studies (65–764 slices), reflecting differenc scan coverage. This variation has little impact on the planned 2D segmentation or classificat pipeline.

- Pixel Spacing Variation:

  Pixel spacing ranges from approximately 0.46 mm to 0.98 mm, indicating substantial differer in-plane spatial resolution. Need for image resampling to ensure consistent physical dimens across all CT scans before model training.

- Slice Thickness and Slice Spacing:

  Slice thickness (0.6–5.0 mm) and slice spacing (0.45–5.0 mm) show significant variability ar studies. These differences lead to inconsistent voxel sizes and further justify resampling to a standardized spatial resolution during preprocessing.

- Scan Length:

  Scan length ranges from 165 mm to 680 mm, showing that CT scans cover different anatom extents. It does not directly affect the preprocessing pipeline for slice-based analysis.

- Rescale Parameters:

The rescale slope is consistently equal to 1, while the rescale intercept is predominantly –10 a few exceptions. Since scan.to_volume() in PyLIDC already converts the images to Hounsf (HU), no additional intensity conversion is required during preprocessing.

- Hounsfield Unit (HU) Distribution:

  The HU statistics (mean, standard deviation, and percentile values) vary substantially acros indicating considerable intensity differences between CT scans. This observation supports t application of HU clipping to remove extreme intensity values and intensity normalization to data consistency before training.

- Reconstruction Diameter:

  The reconstruction diameter varies between 236 mm and 500 mm, reflecting differences in t of view across CT scanners. Because pixel spacing already captures the physical image res reconstruction diameter is informative for dataset characterization but is not directly required preprocessing.

EDA Dashboard

In [ ]:
```
# Dashboard Layout
n_charts = 6
n_cols = 3
n_rows = math.ceil(n_charts / n_cols)

fig, axes = plt.subplots(
    n_rows,
    n_cols,
    figsize=(6 * n_cols, 5 * n_rows)
)

fig.suptitle(
    "CT Geometry Metadata Dashboard",
    fontsize=18,
    fontweight="bold"
)

axes = axes.flatten()

# Number of Slices Distribution
sns.histplot(
    dicom_metadata_df["num_slices"],
    bins=30,
    kde=True,
    ax=axes[0]
)

axes[0].axvline(
    dicom_metadata_df["num_slices"].mean(),
    color="green",
    linestyle="--",
    label="Mean"
)
```

```
axes[0].axvline(
    dicom_metadata_df["num_slices"].median(),
    color="red",
    linestyle="-.",
    label="Median"
)

axes[0].set_title("Number of Slices")
axes[0].set_xlabel("Slices")
axes[0].set_ylabel("Count")
axes[0].legend()

# Slice Thickness Distribution
sns.histplot(
    dicom_metadata_df["slice_thickness_mm"],
    bins="auto",
    kde=True,
    ax=axes[1]
)

axes[1].set_title("Slice Thickness")
axes[1].set_xlabel("Thickness (mm)")
axes[1].set_ylabel("Count")

# Slice Spacing Distribution
sns.histplot(
    dicom_metadata_df["slice_spacing_mm"],
    bins="auto",
    kde=True,
    ax=axes[2]
)

axes[2].set_title("Slice Spacing")
axes[2].set_xlabel("Spacing (mm)")
axes[2].set_ylabel("Count")

# Boxplot: Number of Slices
sns.boxplot(
    y=dicom_metadata_df["num_slices"],
    ax=axes[3]
)

axes[3].set_title("Number of Slices")
axes[3].set_ylabel("Slices")

# Boxplot: Slice Thickness
sns.boxplot(
    y=dicom_metadata_df["slice_thickness_mm"],
    ax=axes[4]
)

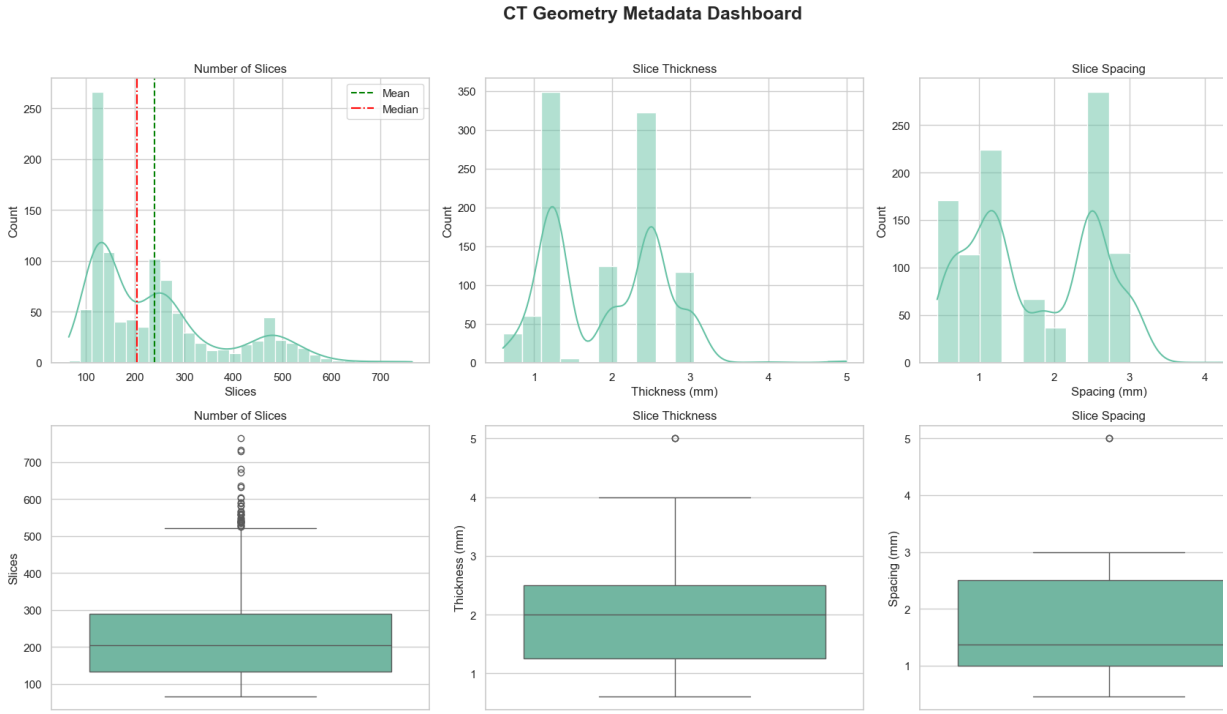
axes[4].set_title("Slice Thickness")
axes[4].set_ylabel("Thickness (mm)")

# Boxplot: Slice Spacing
sns.boxplot(
    y=dicom_metadata_df["slice_spacing_mm"],
    ax=axes[5]
)
```

```
axes[5].set_title("Slice Spacing")
axes[5].set_ylabel("Spacing (mm)")

# Layout
plt.tight_layout(rect=[0, 0, 1, 0.96])

plt.show()
```



Clustering

Run PyLidc Clustering and Extract Information

In [ ]:
```
# Load All Scans
scans = pl.query(pl.Scan).all()

# Extract Cluster Metadata
cluster_rows = []

for patient_idx, scan in enumerate(
    tqdm(scans, desc="Extracting Cluster Metadata")
):
    clusters = scan.cluster_annotations()

    for cluster_idx, cluster in enumerate(clusters):
        # Generate consensus segmentation
        cmask, cbbox, masks = consensus(
            cluster,
            clevel=0.5
        )

        y_slice, x_slice, z_slice = cbbox

        # Consensus Slice Lists

        # All slices covered by the consensus bounding box
```

```
        consensus_bbox_slice_list = list(
            range(z_slice.start, z_slice.stop)
        )

        # Only slices containing at least one consensus voxel
        consensus_mask_slice_list = [
            z_slice.start + local_z
            for local_z in range(cmask.shape[2])
            if np.any(cmask[:, :, local_z])
        ]

        # Bounding Box
        bbox_height = y_slice.stop - y_slice.start
        bbox_width = x_slice.stop - x_slice.start
        bbox_depth = z_slice.stop - z_slice.start

        bbox_volume = (
            bbox_height
            * bbox_width
            * bbox_depth
        )

        # Consensus Statistics
        consensus_voxels = int(cmask.sum())

        mask_density = (
            consensus_voxels / bbox_volume
            if bbox_volume > 0
            else np.nan
        )

        # Bounding Box Center
        centroid_y = (y_slice.start + y_slice.stop) / 2
        centroid_x = (x_slice.start + x_slice.stop) / 2
        centroid_z = (z_slice.start + z_slice.stop) / 2

        # Annotation IDs
        annotation_ids = [
            ann.id
            for ann in cluster
        ]

        # Malignancy
        malignancy = [
            ann.malignancy
            for ann in cluster
        ]

        # Geometry
        diameter = [
            ann.diameter
            for ann in cluster
        ]

        volume = [
            ann.volume
            for ann in cluster
        ]

        surface_area = [
```

```
            ann.surface_area
            for ann in cluster
        ]

        # Semantic Features
        subtlety = [
            ann.subtlety
            for ann in cluster
        ]

        internal_structure = [
            ann.internalStructure
            for ann in cluster
        ]

        calcification = [
            ann.calcification
            for ann in cluster
        ]

        sphericity = [
            ann.sphericity
            for ann in cluster
        ]

        margin = [
            ann.margin
            for ann in cluster
        ]

        lobulation = [
            ann.lobulation
            for ann in cluster
        ]

        spiculation = [
            ann.spiculation
            for ann in cluster
        ]

        texture = [
            ann.texture
            for ann in cluster
        ]

        # Save Metadata
        cluster_rows.append({
            # Study Information

            "patient_idx": patient_idx,
            "patient_id": scan.patient_id,
            "study_instance_uid": scan.study_instance_uid,
            "series_instance_uid": scan.series_instance_uid,

            # Cluster Information

            "cluster_id": cluster_idx,
            "cluster_uid":
                f"{scan.patient_id}_cluster_{cluster_idx}",
```

```
            "num_annotations": len(cluster),
            "annotation_ids": annotation_ids,

            # Consensus Information

            # Dimensions of the consensus mask
            "consensus_num_slices": cmask.shape[2],
            "consensus_height": cmask.shape[0],
            "consensus_width": cmask.shape[1],

            # Slice lists
            "consensus_bbox_slice_list":
                consensus_bbox_slice_list,

            "consensus_mask_slice_list":
                consensus_mask_slice_list,

            # Number of slices containing consensus voxels
            "consensus_mask_num_slices":
                len(consensus_mask_slice_list),

            # Voxel statistics
            "consensus_voxels":
                consensus_voxels,
            "consensus_bbox_volume":
                bbox_volume,
            "consensus_mask_density":
                mask_density,

            # Bounding Box
            "bbox_y_min": y_slice.start,
            "bbox_y_max": y_slice.stop,
            "bbox_x_min": x_slice.start,
            "bbox_x_max": x_slice.stop,
            "bbox_z_min": z_slice.start,
            "bbox_z_max": z_slice.stop,
            "bbox_height": bbox_height,
            "bbox_width": bbox_width,
            "bbox_depth": bbox_depth,

            # Bounding Box Center
            "centroid_y": centroid_y,
            "centroid_x": centroid_x,
            "centroid_z": centroid_z,

            # Malignancy
            "malignancy_scores": malignancy,
            "malignancy_mean":
                np.mean(malignancy),
            "malignancy_std":
                np.std(malignancy),
            "malignancy_min":
                np.min(malignancy),
            "malignancy_max":
                np.max(malignancy),
            "malignancy_agreement":
                len(set(malignancy)) == 1,

            # Geometry
            "diameter_mean":
```

```
                np.mean(diameter),
            "diameter_std":
                np.std(diameter),
            "diameter_min":
                np.min(diameter),
            "diameter_max":
                np.max(diameter),
            "volume_mean":
                np.mean(volume),
            "volume_std":
                np.std(volume),
            "volume_min":
                np.min(volume),
            "volume_max":
                np.max(volume),
            "surface_area_mean":
                np.mean(surface_area),
            "surface_area_std":
                np.std(surface_area),
            "surface_area_min":
                np.min(surface_area),
            "surface_area_max":
                np.max(surface_area),

            # Semantic Features
            "subtlety_mean":
                np.mean(subtlety),
            "internal_structure_mean":
                np.mean(internal_structure),
            "calcification_mean":
                np.mean(calcification),
            "sphericity_mean":
                np.mean(sphericity),
            "margin_mean":
                np.mean(margin),
            "lobulation_mean":
                np.mean(lobulation),
            "spiculation_mean":
                np.mean(spiculation),
            "texture_mean":
                np.mean(texture),
        })

# Create DataFrame
cluster_df = pd.DataFrame(cluster_rows)

# Save CSV
csv_path = "../../000_dataset/_lidc/000_metadata/001_cluster_metadata.csv

# cluster_df.to_csv(
#     csv_path,
#     index=False
# )

print("=" * 60)
print(f"Total Patients : {cluster_df['patient_id'].nunique()}")
print(f"Total Clusters : {len(cluster_df)}")
print(f"Target CSV path: {csv_path} (saving disabled)")
print("=" * 60)
```

```
display(cluster_df.head())
```

```
Extracting Cluster Metadata:   6%|▌         | 66/1018 [00:50<16:50,  1.06s/it
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  10%|▉         | 99/1018 [01:19<12:09,  1.26it/s
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  14%|█▎        | 140/1018 [01:52<15:52,  1.08s/i
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  20%|█▉        | 205/1018 [02:43<08:10,  1.66it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  25%|██▍       | 252/1018 [03:05<05:51,  2.18it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  33%|███       | 333/1018 [03:44<04:50,  2.36it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  33%|███       | 341/1018 [03:49<06:02,  1.87it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  36%|███▍      | 370/1018 [04:10<04:51,  2.22it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  40%|███▉      | 407/1018 [04:37<10:27,  1.03s/i
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  60%|█████▉    | 612/1018 [07:01<06:14,  1.08it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  70%|██████▉   | 712/1018 [08:05<01:38,  3.10it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  78%|███████▊  | 790/1018 [08:55<03:50,  1.01s/i
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  78%|███████▊  | 791/1018 [08:56<03:02,  1.25it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata:  83%|████████▎ | 840/1018 [09:38<01:37,  1.82it/
Failed to reduce all groups to <= 4 Annotations.
Some nodules may be close and must be grouped manually.
Extracting Cluster Metadata: 100%|██████████| 1018/1018 [11:52<00:00,  1.43it
============================================================
Total Patients : 875
Total Clusters : 2651
Target CSV path: ../../000_dataset/_lidc/000_metadata/001_cluster_metadata.cs
(saving disabled)
============================================================
```

	patient_idx	patient_id	study_instance_uid	
0	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
1	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
2	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
3	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
4	1	LIDC-IDRI-0069	1.3.6.1.4.1.14519.5.2.1.6279.6001.303241414168...	1.3.6.1.4.1.14519.5.

5 rows × 55 columns

Load Data

In [ ]:
```
cluster_metadata_df = pd.read_csv("../../000_dataset/_lidc/000_metadata/0

print(f"{'='*20} DataFrame Information {'='*20}")
cluster_metadata_df.info()
print(f"\n{'='*20} First 5 Rows Of A DataFrame {'='*20}")
display(cluster_metadata_df.head())
```

```
==================== DataFrame Information ====================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2651 entries, 0 to 2650
Data columns (total 55 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   patient_idx                 2651 non-null   int64
 1   patient_id                  2651 non-null   object
 2   study_instance_uid          2651 non-null   object
 3   series_instance_uid         2651 non-null   object
 4   cluster_id                  2651 non-null   int64
 5   cluster_uid                 2651 non-null   object
 6   num_annotations             2651 non-null   int64
 7   annotation_ids              2651 non-null   object
 8   consensus_num_slices        2651 non-null   int64
 9   consensus_height            2651 non-null   int64
 10  consensus_width             2651 non-null   int64
 11  consensus_bbox_slice_list   2651 non-null   object
 12  consensus_mask_slice_list   2651 non-null   object
 13  consensus_mask_num_slices   2651 non-null   int64
 14  consensus_voxels            2651 non-null   int64
 15  consensus_bbox_volume       2651 non-null   int64
 16  consensus_mask_density      2651 non-null   float64
 17  bbox_y_min                  2651 non-null   int64
 18  bbox_y_max                  2651 non-null   int64
 19  bbox_x_min                  2651 non-null   int64
 20  bbox_x_max                  2651 non-null   int64
 21  bbox_z_min                  2651 non-null   int64
 22  bbox_z_max                  2651 non-null   int64
 23  bbox_height                 2651 non-null   int64
 24  bbox_width                  2651 non-null   int64
 25  bbox_depth                  2651 non-null   int64
 26  centroid_y                  2651 non-null   float64
 27  centroid_x                  2651 non-null   float64
 28  centroid_z                  2651 non-null   float64
 29  malignancy_scores           2651 non-null   object
 30  malignancy_mean             2651 non-null   float64
 31  malignancy_std              2651 non-null   float64
 32  malignancy_min              2651 non-null   int64
 33  malignancy_max              2651 non-null   int64
 34  malignancy_agreement        2651 non-null   bool
 35  diameter_mean               2651 non-null   float64
 36  diameter_std                2651 non-null   float64
 37  diameter_min                2651 non-null   float64
 38  diameter_max                2651 non-null   float64
 39  volume_mean                 2651 non-null   float64
 40  volume_std                  2651 non-null   float64
 41  volume_min                  2651 non-null   float64
 42  volume_max                  2651 non-null   float64
 43  surface_area_mean           2651 non-null   float64
 44  surface_area_std            2651 non-null   float64
 45  surface_area_min            2651 non-null   float64
 46  surface_area_max            2651 non-null   float64
 47  subtlety_mean               2651 non-null   float64
 48  internal_structure_mean     2651 non-null   float64
 49  calcification_mean          2651 non-null   float64
 50  sphericity_mean             2651 non-null   float64
 51  margin_mean                 2651 non-null   float64
 52  lobulation_mean             2651 non-null   float64
 53  spiculation_mean            2651 non-null   float64
```

```
 54  texture_mean                2651 non-null   float64
dtypes: bool(1), float64(26), int64(20), object(8)
memory usage: 1.1+ MB

==================== First 5 Rows Of A DataFrame ====================
```

	patient_idx	patient_id	study_instance_uid	
0	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
1	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
2	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
3	0	LIDC-IDRI-0078	1.3.6.1.4.1.14519.5.2.1.6279.6001.339170810277...	1.3.6.1.4.1.14519.5.
4	1	LIDC-IDRI-0069	1.3.6.1.4.1.14519.5.2.1.6279.6001.303241414168...	1.3.6.1.4.1.14519.5.

5 rows × 55 columns

Descriptive Statistics

In [ ]:
```
cluster_metadata_df.describe()
```

Out[ ]:

	patient_idx	cluster_id	num_annotations	consensus_num_slices	consensus_he
count	2651.000000	2651.000000	2651.000000	2651.000000	2651.00
mean	505.601283	2.158431	2.587326	5.961901	15.81
std	301.336148	2.779538	1.249986	5.278048	10.51
min	0.000000	0.000000	1.000000	1.000000	5.00
25%	230.000000	0.000000	1.000000	3.000000	9.00
50%	497.000000	1.000000	3.000000	4.000000	12.00
75%	783.000000	3.000000	4.000000	7.000000	17.00
max	1017.000000	22.000000	8.000000	57.000000	98.00

8 rows × 46 columns

Cluster Overview

In [ ]:
```
# Cluster Overview
overview = pd.DataFrame({
    "Metric": [
        "Total Patients",
        "Total Studies",
        "Total Clusters",
        "Total Annotations",
        "Average Annotations per Cluster",
        "Minimum Annotations per Cluster",
        "Maximum Annotations per Cluster",
    ],
    "Value": [
```

```
        cluster_metadata_df["patient_id"].nunique(),
        cluster_metadata_df["study_instance_uid"].nunique(),
        len(cluster_metadata_df),
        cluster_metadata_df["num_annotations"].sum(),
        round(cluster_metadata_df["num_annotations"].mean(), 2),
        cluster_metadata_df["num_annotations"].min(),
        cluster_metadata_df["num_annotations"].max(),
    ]
})

display(overview)
```

	Metric	Value
0	Total Patients	875.00
1	Total Studies	883.00
2	Total Clusters	2651.00
3	Total Annotations	6859.00
4	Average Annotations per Cluster	2.59
5	Minimum Annotations per Cluster	1.00
6	Maximum Annotations per Cluster	8.00

In [ ]:
```
# Verify Duplicate Studies in Cluster Metadata
duplicate_pairs = duplicate_ids_ct_df[
    ["patient_id", "study_instance_uid"]
].drop_duplicates()

cluster_pairs = cluster_metadata_df[
    ["patient_id", "study_instance_uid"]
].drop_duplicates()

verification = duplicate_pairs.merge(
    cluster_pairs,
    on=["patient_id", "study_instance_uid"],
    how="left",
    indicator=True,
)

missing = verification[verification["_merge"] == "left_only"]

print(f"Duplicate studies in CT metadata    : {len(duplicate_pairs)}")
print(f"Found in cluster metadata           : {(verification['_merge'] ==
print(f"Missing                             : {len(missing)}")
```

```
Duplicate studies in CT metadata    : 16
Found in cluster metadata           : 16
Missing                             : 0
```

- Out of 1,010 unique patients, 875 patients contained at least one clustered pulmonary nodu whereas 135 patients had no clustered nodules.
- A total of 883 CT studies produced clustered nodules, preserving the original difference of e additional studies due to patients with multiple CT examinations.
- Therefore, the clustering process excluded only studies without clustered nodules and did n the original patient–study relationship.

Number of Cluster per Study

In [ ]:
```
study_cluster_counts = (
    cluster_metadata_df
    .groupby(
        ["study_instance_uid", "patient_id"]
    )
    .size()
    .reset_index(name="num_clusters")
)

study_cluster_counts["num_clusters"].describe()
```

Out[ ]:
```
count    883.000000
mean       3.002265
std        2.637596
min        1.000000
25%        1.000000
50%        2.000000
75%        4.000000
max       23.000000
Name: num_clusters, dtype: float64
```

Distribution of the number of clusters per study

In [ ]:
```
distribution = (
    study_cluster_counts["num_clusters"]
    .value_counts()
    .sort_index()
)

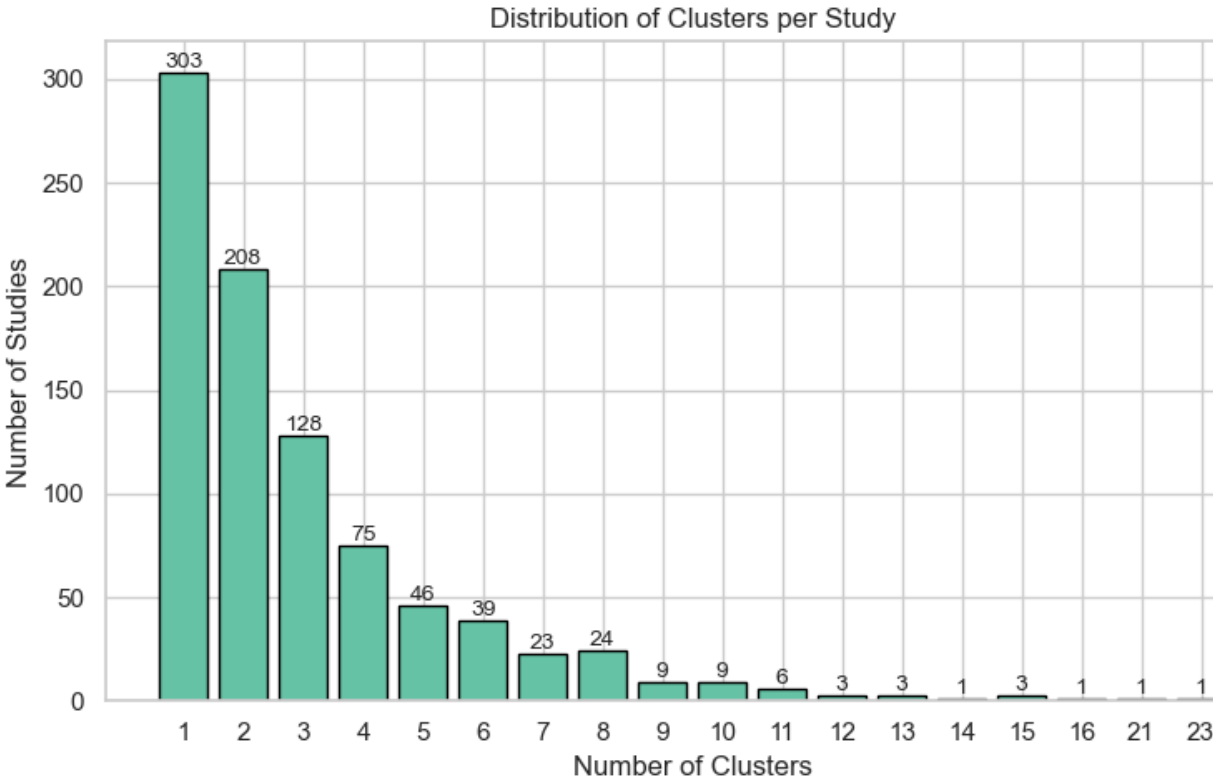
fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(
    distribution.index.astype(str),
    distribution.values,
    edgecolor="black"
)

ax.set_title("Distribution of Clusters per Study")
ax.set_xlabel("Number of Clusters")
ax.set_ylabel("Number of Studies")

# Add value labels
for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height}",
        ha="center",
        va="bottom",
        fontsize=10,
    )

plt.tight_layout()
plt.show()
```



Studies without Clusters

In [ ]:
```
studies_without_clusters = (
    ct_df[
        ~ct_df["study_instance_uid"].isin(
            cluster_metadata_df["study_instance_uid"]
        )
    ][
        ["patient_id", "study_instance_uid"]
    ]
    .sort_values(["patient_id", "study_instance_uid"])
    .reset_index(drop=True)
)

display(studies_without_clusters)
```

	patient_id	study_instance_uid
0	LIDC-IDRI-0028	1.3.6.1.4.1.14519.5.2.1.6279.6001.313822485916...
1	LIDC-IDRI-0032	1.3.6.1.4.1.14519.5.2.1.6279.6001.188500211166...
2	LIDC-IDRI-0062	1.3.6.1.4.1.14519.5.2.1.6279.6001.121427641977...
3	LIDC-IDRI-0071	1.3.6.1.4.1.14519.5.2.1.6279.6001.194873495479...
4	LIDC-IDRI-0100	1.3.6.1.4.1.14519.5.2.1.6279.6001.318037728418...
...	...	...
130	LIDC-IDRI-0975	1.3.6.1.4.1.14519.5.2.1.6279.6001.238406803970...
131	LIDC-IDRI-0979	1.3.6.1.4.1.14519.5.2.1.6279.6001.739936498619...
132	LIDC-IDRI-0988	1.3.6.1.4.1.14519.5.2.1.6279.6001.127337418291...
133	LIDC-IDRI-0992	1.3.6.1.4.1.14519.5.2.1.6279.6001.533848162484...
134	LIDC-IDRI-0995	1.3.6.1.4.1.14519.5.2.1.6279.6001.111762382184...

135 rows × 2 columns

EDA Dashboard - Cluster Geometry

In [ ]:
```python
# Geometry features to visualize
geometry_features = [
    ("diameter_mean", "Diameter (mm)"),
    ("volume_mean", "Volume (mm³)"),
    ("surface_area_mean", "Surface Area (mm²)"),
    # ("consensus_bbox_volume", "Bounding Box Volume (voxels)"),
    # ("consensus_voxels", "Consensus Voxels"),
    # ("consensus_mask_density", "Mask Density"),
]

# Features displayed using logarithmic scale
log_scale_features = {
    "volume_mean",
    "surface_area_mean",
    # "consensus_bbox_volume",
    # "consensus_voxels",
}

# Create dashboard layout
n_cols = 3
n_rows = math.ceil(len(geometry_features) / n_cols)

fig, axes = plt.subplots(
    n_rows,
    n_cols,
    figsize=(18, 6)
)

axes = axes.flatten()

# Plot each geometry feature
for ax, (column, title) in zip(axes, geometry_features):

    # Remove missing values
```

```
    data = cluster_metadata_df[column].dropna()

    # Plot histogram
    ax.hist(
        data,
        bins=30,
        color="#69b3a2",
        edgecolor="black",
        alpha=0.85
    )

    # Apply logarithmic x-axis for skewed features
    if column in log_scale_features:
        ax.set_xscale("log")

    # Calculate summary statistics
    mean = data.mean()
    median = data.median()

    # Draw mean line
    ax.axvline(
        mean,
        color="red",
        linestyle="--",
        linewidth=2,
        label="Mean"
    )

    # Draw median line
    ax.axvline(
        median,
        color="green",
        linestyle=":",
        linewidth=2,
        label="Median"
    )

    # Configure subplot
    ax.set_title(
        title,
        fontsize=12,
        fontweight="bold"
    )

    ax.set_xlabel(title)
    ax.set_ylabel("Frequency")

    ax.grid(
        linestyle="--",
        alpha=0.4
    )

    # Display legend
    ax.legend(
        loc="upper right",
        fontsize=9,
        frameon=True
    )

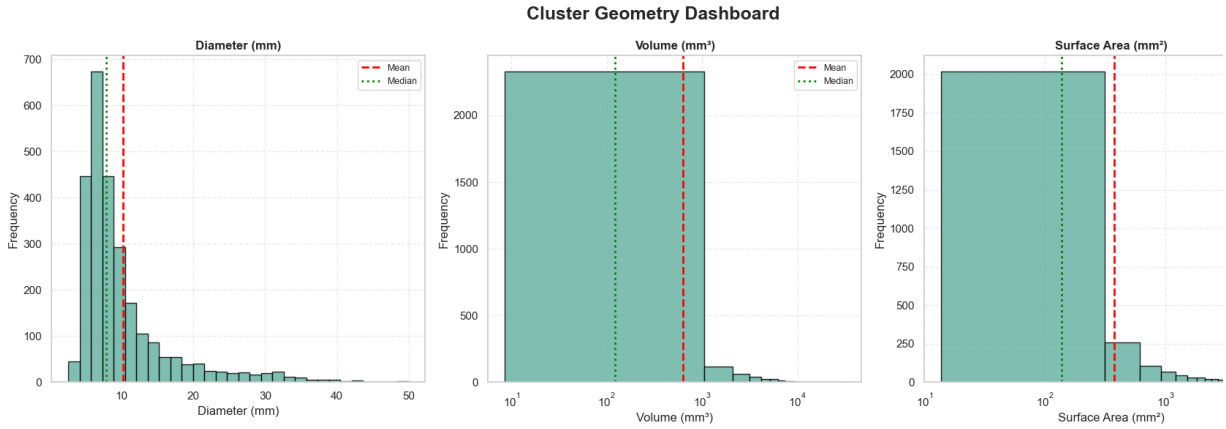
# Hide unused subplot(s)
```

```
for i in range(len(geometry_features), len(axes)):
    axes[i].axis("off")

# Figure title
plt.suptitle(
    "Cluster Geometry Dashboard",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()

plt.show()
```



In [ ]:
```
# Cluster Geometry Summary Statistics
geometry_columns = [
    "diameter_mean",
    "volume_mean",
    "surface_area_mean",
    "consensus_bbox_volume",
    "consensus_voxels",
    "consensus_mask_density",
]

display(
    cluster_metadata_df[geometry_columns]
    .describe()
    .round(2)
    .T
)
```

	count	mean	std	min	25%	50%	75%	r
diameter_mean	2651.0	10.16	6.73	2.59	6.13	7.82	11.14	49
volume_mean	2651.0	630.33	1779.43	8.54	63.63	122.42	323.06	31112
surface_area_mean	2651.0	375.66	695.36	13.76	84.09	137.67	299.77	9026
consensus_bbox_volume	2651.0	4115.00	13562.96	30.00	288.00	624.00	1900.00	229824
consensus_voxels	2651.0	767.75	2146.46	5.00	74.00	148.00	417.50	45329
consensus_mask_density	2651.0	0.25	0.08	0.00	0.19	0.25	0.30	0

In [ ]:
```
# Count clusters based on the mean diameter threshold
diameter_threshold_table = (
    cluster_metadata_df["diameter_mean"]
```

```
    .lt(3)
    .map({
        True: "< 3 mm",
        False: "≥ 3 mm"
    })
    .value_counts()
    .reindex(["< 3 mm", "≥ 3 mm"])
    .rename_axis("Mean Diameter")
    .reset_index(name="Number of Clusters")
)

# Calculate percentage
diameter_threshold_table["Percentage (%)"] = (
    diameter_threshold_table["Number of Clusters"]
    / diameter_threshold_table["Number of Clusters"].sum()
    * 100
).round(2)

# Display table
display(diameter_threshold_table)
```

	Mean Diameter	Number of Clusters	Percentage (%)
0	< 3 mm	3	0.11
1	≥ 3 mm	2648	99.89

Among the 2,651 extracted clusters, 2,648 clusters (99.89%) had a mean diameter of at least 3 r
while only three clusters (0.11%) had a mean diameter below 3 mm. Given the negligible proport
clusters with a mean diameter below 3 mm and the inherent variability in diameter measurement
radiologists, no further diameter-based data selection was performed.

EDA Dashboard - Consensus

In [ ]:
```
# Consensus features to visualize
consensus_features = [
    ("consensus_num_slices", "Consensus Slices"),
    ("consensus_height", "Consensus Height (pixels)"),
    ("consensus_width", "Consensus Width (pixels)"),
    ("consensus_voxels", "Consensus Voxels"),
    #("bbox_depth", "Bounding Box Depth (slices)"),
    ("consensus_mask_density", "Mask Density"),
]

# Features displayed using logarithmic scale
log_scale_features = {
    "consensus_voxels",
}

# Create dashboard layout
n_cols = 3
n_rows = math.ceil(len(consensus_features) / n_cols)

fig, axes = plt.subplots(
    n_rows,
    n_cols,
    figsize=(18, 10)
```

```
)

axes = axes.flatten()

# Plot each consensus feature
for ax, (column, title) in zip(axes, consensus_features):

    # Remove missing values
    data = cluster_metadata_df[column].dropna()

    # Plot histogram
    ax.hist(
        data,
        bins=30,
        color="#5DADE2",
        edgecolor="black",
        alpha=0.85
    )

    # Apply logarithmic scale
    if column in log_scale_features:
        ax.set_xscale("log")

    # Calculate statistics
    mean = data.mean()
    median = data.median()

    # Mean
    ax.axvline(
        mean,
        color="red",
        linestyle="--",
        linewidth=2,
        label="Mean"
    )

    # Median
    ax.axvline(
        median,
        color="green",
        linestyle=":",
        linewidth=2,
        label="Median"
    )

    # Configure subplot
    ax.set_title(
        title,
        fontsize=12,
        fontweight="bold"
    )

    ax.set_xlabel(title)
    ax.set_ylabel("Frequency")

    ax.grid(
        linestyle="--",
        alpha=0.4
    )
```

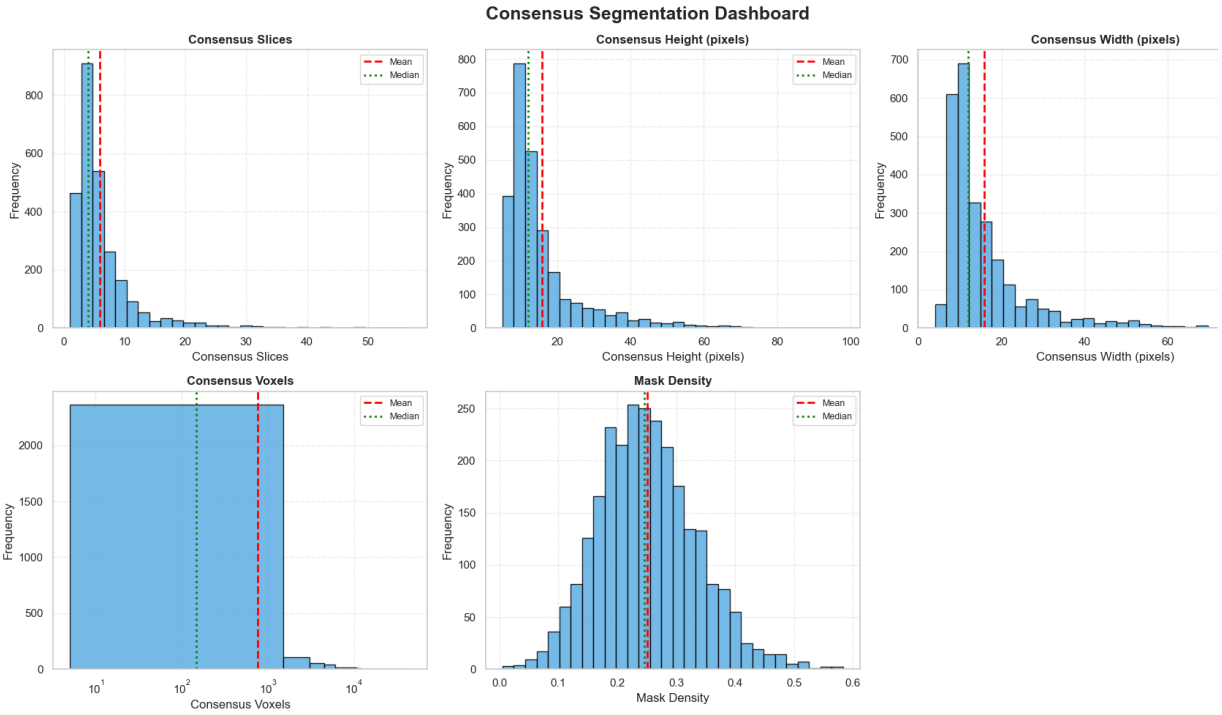
```
    ax.legend(
        loc="upper right",
        fontsize=9
    )

# Hide unused subplot(s)
for i in range(len(consensus_features), len(axes)):
    axes[i].axis("off")

# Figure title
plt.suptitle(
    "Consensus Segmentation Dashboard",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()

plt.show()
```



EDA Dashboard - Malignancy

In [ ]:
```
# Create dashboard layout
fig, axes = plt.subplots(
    2,
    3,
    figsize=(18, 10)
)

axes = axes.flatten()

# Mean Malignancy
data = cluster_metadata_df["malignancy_mean"]

axes[0].hist(
    data,
    bins=8,
```

```
    edgecolor="black",
    color="#6baed6"
)

axes[0].axvline(
    data.mean(),
    color="red",
    linestyle="--",
    linewidth=2,
    label="Mean"
)

axes[0].axvline(
    data.median(),
    color="green",
    linestyle=":",
    linewidth=2,
    label="Median"
)

axes[0].set_title("Mean Malignancy")
axes[0].set_xlabel("Mean Score")
axes[0].set_ylabel("Frequency")
axes[0].grid(alpha=0.3)
axes[0].legend()

# Malignancy Standard Deviation
data = cluster_metadata_df["malignancy_std"]

axes[1].hist(
    data,
    bins=20,
    edgecolor="black",
    color="#74c476"
)

axes[1].axvline(
    data.mean(),
    color="red",
    linestyle="--",
    linewidth=2,
    label="Mean"
)

axes[1].axvline(
    data.median(),
    color="green",
    linestyle=":",
    linewidth=2,
    label="Median"
)

axes[1].set_title("Malignancy Standard Deviation")
axes[1].set_xlabel("Standard Deviation")
axes[1].set_ylabel("Frequency")
axes[1].grid(alpha=0.3)
axes[1].legend()

# Malignancy Agreement
agreement = (
```

```
    cluster_metadata_df["malignancy_agreement"]
    .map({
        True: "Agreement",
        False: "Disagreement"
    })
    .value_counts()
)

bars = axes[2].bar(
    agreement.index,
    agreement.values,
    edgecolor="black",
    color=["#2ca25f", "#fc9272"]
)

for bar in bars:

    axes[2].text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height(),
        int(bar.get_height()),
        ha="center",
        va="bottom",
        fontweight="bold"
    )

axes[2].set_title("Radiologist Agreement")
axes[2].set_ylabel("Number of Clusters")
axes[2].grid(axis="y", alpha=0.3)

# Minimum Malignancy
minimum = (
    cluster_metadata_df["malignancy_min"]
    .value_counts()
    .sort_index()
)

axes[3].bar(
    minimum.index.astype(str),
    minimum.values,
    edgecolor="black",
    color="#9ecae1"
)

axes[3].set_title("Minimum Malignancy Score")
axes[3].set_xlabel("Score")
axes[3].set_ylabel("Number of Clusters")
axes[3].grid(axis="y", alpha=0.3)

# Maximum Malignancy
maximum = (
    cluster_metadata_df["malignancy_max"]
    .value_counts()
    .sort_index()
)

axes[4].bar(
    maximum.index.astype(str),
    maximum.values,
    edgecolor="black",
```

```
    color="#fdae6b"
)

axes[4].set_title("Maximum Malignancy Score")
axes[4].set_xlabel("Score")
axes[4].set_ylabel("Number of Clusters")
axes[4].grid(axis="y", alpha=0.3)

# Mean Malignancy Category
def malignancy_category(score):
    if score <= 2.5:
        return "Benign"
    elif score >= 3.5:
        return "Malignant"

    return "Indeterminate"

category = (
    cluster_metadata_df["malignancy_mean"]
    .apply(malignancy_category)
    .value_counts()
    .reindex(["Benign", "Indeterminate", "Malignant"])
)

bars = axes[5].bar(
    category.index,
    category.values,
    edgecolor="black",
    color=["#66c2a4", "#ffd92f", "#fc8d62"]
)

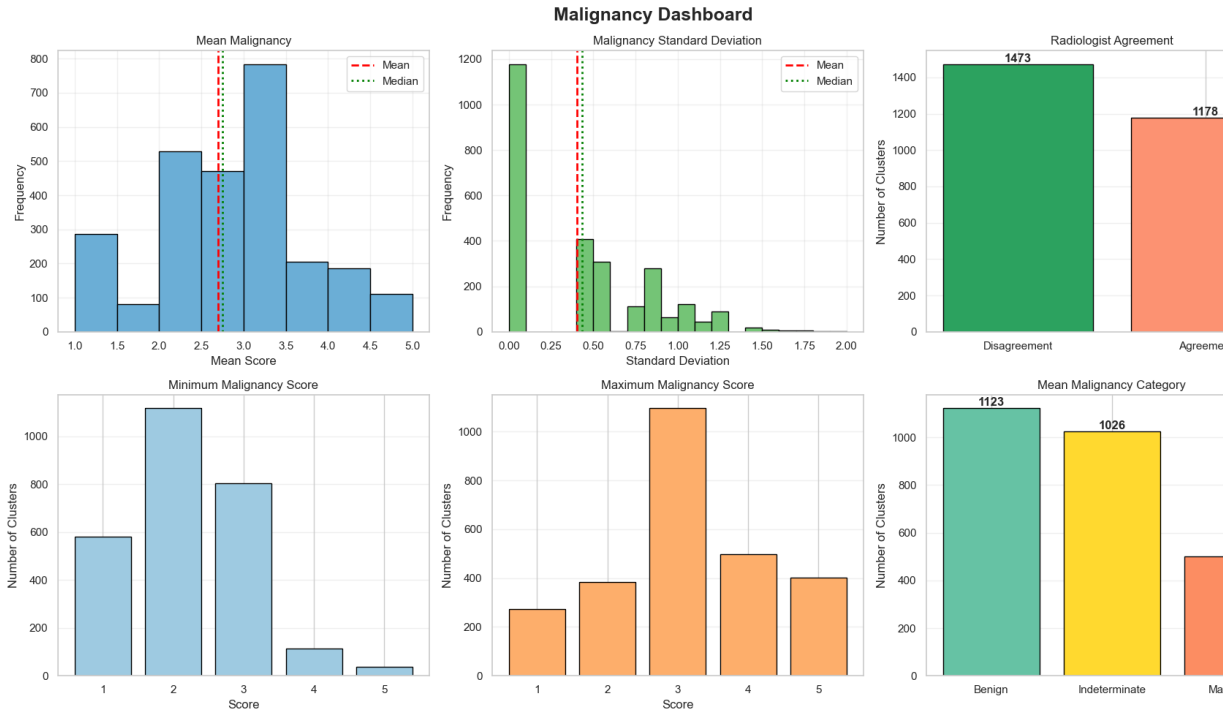
for bar in bars:
    axes[5].text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height(),
        int(bar.get_height()),
        ha="center",
        va="bottom",
        fontweight="bold"
    )

axes[5].set_title("Mean Malignancy Category")
axes[5].set_ylabel("Number of Clusters")
axes[5].grid(axis="y", alpha=0.3)


# Figure title
plt.suptitle(
    "Malignancy Dashboard",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()

plt.show()
```



- The mean malignancy scores are concentrated between 2.5 and 3.5, indicating that most nc were assigned intermediate malignancy ratings by the radiologists. Fewer nodules received (1–2) or very high (4–5) mean malignancy scores.
- Most clusters have a low standard deviation, indicating that radiologists generally showed gc agreement when assigning malignancy scores. However, some clusters exhibit higher variab reflecting differences in radiologists' assessments.
- Out of 2,651 clusters, 1,178 (44.44%) showed complete agreement among radiologists, whi (55.56%) showed some level of disagreement. This indicates that variability in malignancy assessment is common in the LIDC-IDRI dataset.
- Most clusters have a minimum malignancy score of 2 or 3, suggesting that at least one radic frequently assigned a low to intermediate malignancy rating. Only a small number of clusters received the highest minimum score of 5, indicating unanimous high suspicion of malignanc
- The maximum malignancy score is most frequently 3, followed by 4 and 5, indicating that ma clusters were considered moderately suspicious by at least one radiologist. Relatively few cl had a maximum score of 1, showing that completely benign assessments were less commo
- Based on the mean malignancy score, the dataset contains 1,123 benign, 1,026 indetermina 502 malignant clusters. The large number of indeterminate nodules supports the need for a selection step that excludes ambiguous cases before model development.

Data Selection

In [ ]:
```python
def assign_malignancy_label(mean_score):
    """
    Assign a malignancy label based on the mean malignancy score.
    """

    if mean_score <= 2.5:
        return "Benign"
```

```
    elif mean_score >= 3.5:
        return "Malignant"

    else:
        return "Indeterminate"


# Add malignancy label
cluster_metadata_df["label"] = (
    cluster_metadata_df["malignancy_mean"]
    .apply(assign_malignancy_label)
)

# Display label distribution
label_distribution = (
    cluster_metadata_df["label"]
    .value_counts()
    .reindex(["Benign", "Indeterminate", "Malignant"])
    .rename_axis("Label")
    .reset_index(name="Number of Clusters")
)

label_distribution["Percentage (%)"] = (
    label_distribution["Number of Clusters"]
    / len(cluster_metadata_df)
    * 100
).round(2)

display(label_distribution)
```

	Label	Number of Clusters	Percentage (%)
0	Benign	1123	42.36
1	Indeterminate	1026	38.70
2	Malignant	502	18.94

Remove Duplicate Studies

```
    excluded_study_uids = [

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.218658642102832118810712329678",
    LIDC-IDRI-0132

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.222444895275634974977432374937",
    LIDC-IDRI-0151

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.709924281509506265799179464374",
    LIDC-IDRI-0315

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.124508094927772723365648607398",
    LIDC-IDRI-0332

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.221138977753587043062153841454",
    LIDC-IDRI-0355
```

```
    "1.3.6.1.4.1.14519.5.2.1.6279.6001.216207548522622026268886920069",
    LIDC-IDRI-0365

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.265657827498396379702226874113",
    LIDC-IDRI-0442

    "1.3.6.1.4.1.14519.5.2.1.6279.6001.202012743926377385386645171009",
    LIDC-IDRI-0484
    ]
```

In [ ]:
```python
num_clusters_before = len(cluster_metadata_df)

# Remove duplicate studies
cluster_metadata_df = cluster_metadata_df[
    ~cluster_metadata_df["study_instance_uid"].isin(excluded_study_uids)
].reset_index(drop=True)

# Dataset size after filtering
num_clusters_after = len(cluster_metadata_df)
num_removed = num_clusters_before - num_clusters_after

# Display summary
print(f"Clusters before filtering : {num_clusters_before:,}")
print(f"Clusters removed          : {num_removed:,}")
print(f"Clusters after filtering  : {num_clusters_after:,}")
```

```
Clusters before filtering : 2,651
Clusters removed          : 21
Clusters after filtering  : 2,630
```

Remove Nodules Annotated by Fewer Than Three Radiologists

In [ ]:
```python
num_clusters_before = len(cluster_metadata_df)

# Remove indeterminate nodules
cluster_metadata_df = cluster_metadata_df[
    cluster_metadata_df["label"] != "Indeterminate"
].reset_index(drop=True)

# Dataset size after filtering
num_clusters_after = len(cluster_metadata_df)
num_removed = num_clusters_before - num_clusters_after

# Display filtering summary
print(f"Clusters before filtering : {num_clusters_before:,}")
print(f"Clusters removed          : {num_removed:,}")
print(f"Clusters after filtering  : {num_clusters_after:,}")

# Display label distribution
label_distribution = (
    cluster_metadata_df["label"]
    .value_counts()
    .reindex(["Benign", "Malignant"])
    .rename_axis("Label")
    .reset_index(name="Number of Clusters")
)
```

```
label_distribution["Percentage (%)"] = (
    label_distribution["Number of Clusters"]
    / len(cluster_metadata_df)
    * 100
).round(2)

display(label_distribution)
```

```
Clusters before filtering : 2,630
Clusters removed          : 1,017
Clusters after filtering  : 1,613
```

	Label	Number of Clusters	Percentage (%)
0	Benign	1119	69.37
1	Malignant	494	30.63

Remove Indeterminate Malignanacy

In [ ]:
```
# Dataset size before filtering
num_clusters_before = len(cluster_metadata_df)

# Remove nodules with fewer than three annotations
cluster_metadata_df = cluster_metadata_df[
    cluster_metadata_df["num_annotations"] >= 3
].reset_index(drop=True)

# Dataset size after filtering
num_clusters_after = len(cluster_metadata_df)
num_removed = num_clusters_before - num_clusters_after

# Display filtering summary
print(f"Clusters before filtering : {num_clusters_before:,}")
print(f"Clusters removed          : {num_removed:,}")
print(f"Clusters after filtering  : {num_clusters_after:,}")

# Display annotation distribution
annotation_distribution = (
    cluster_metadata_df["num_annotations"]
    .value_counts()
    .sort_index()
    .rename_axis("Number of Radiologists")
    .reset_index(name="Number of Clusters")
)

annotation_distribution["Percentage (%)"] = (
    annotation_distribution["Number of Clusters"]
    / len(cluster_metadata_df)
    * 100
).round(2)

display(annotation_distribution)
```

```
Clusters before filtering : 1,613
Clusters removed          : 811
Clusters after filtering  : 802
```

	Number of Radiologists	Number of Clusters	Percentage (%)
0	3	225	28.05
1	4	568	70.82
2	5	6	0.75
3	6	1	0.12
4	7	2	0.25

In [ ]:
```
# cluster_metadata_df.to_csv(
#     "../../000_dataset/_lidc/000_metadata/002_cluster_metadata_cleaned.
#     index=False
# )
```

Analysis of Multiple Nodules in a Single Slice

A physical CT slice is identified by `study_instance_uid`, `series_instance_uid`, and `slice_index`. The slice indices are taken from `consensus_mask_slice_list`, so only s containing consensus-mask foreground are counted. Distinct `cluster_id` values are used to duplicate records from inflating the nodule count.

In [13]:
```
from ast import literal_eval

multi_nodule_metadata_path = (
    "../../000_dataset/_lidc/000_metadata/"
    "002_cluster_metadata_cleaned.csv"
)

slice_cluster_df = pd.read_csv(
    multi_nodule_metadata_path,
    usecols=[
        "patient_id",
        "study_instance_uid",
        "series_instance_uid",
        "cluster_id",
        "consensus_mask_slice_list",
    ],
)


def parse_slice_list(value):
    """Parse one serialized consensus slice list safely."""
    if pd.isna(value):
        return []

    parsed = literal_eval(value) if isinstance(value, str) else value

    if not isinstance(parsed, (list, tuple, set)):
        raise ValueError(f"Expected a slice list, received: {parsed!r}")

    return [int(slice_index) for slice_index in parsed]


slice_cluster_df["slice_index"] = (
```

```
    slice_cluster_df["consensus_mask_slice_list"]
    .apply(parse_slice_list)
)

slice_cluster_df = (
    slice_cluster_df
    .explode("slice_index")
    .dropna(subset=["slice_index"])
    .assign(slice_index=lambda data: data["slice_index"].astype(int))
    .drop_duplicates(
        subset=[
            "study_instance_uid",
            "series_instance_uid",
            "slice_index",
            "cluster_id",
        ]
    )
)
```

In [14]:

```
slice_keys = [
    "patient_id",
    "study_instance_uid",
    "series_instance_uid",
    "slice_index",
]

slice_nodule_counts = (
    slice_cluster_df
    .groupby(slice_keys, as_index=False)
    .agg(
        nodule_count=("cluster_id", "nunique"),
        cluster_ids=(
            "cluster_id",
            lambda values: sorted(values.unique().tolist()),
        ),
    )
    .sort_values(
        ["nodule_count", "patient_id", "slice_index"],
        ascending=[False, True, True],
    )
)

multi_nodule_slices = (
    slice_nodule_counts
    .query("nodule_count > 1")
    .reset_index(drop=True)
)

total_nodule_slices = len(slice_nodule_counts)
total_multi_nodule_slices = len(multi_nodule_slices)
multi_nodule_percentage = (
    100 * total_multi_nodule_slices / total_nodule_slices
    if total_nodule_slices
    else 0.0
)
max_nodules_per_slice = (
    int(slice_nodule_counts["nodule_count"].max())
    if total_nodule_slices
    else 0
)
```

```
multi_nodule_scan_count = (
    multi_nodule_slices[
        ["study_instance_uid", "series_instance_uid"]
    ]
    .drop_duplicates()
    .shape[0]
)

print(f"Slices containing at least one nodule : {total_nodule_slices:,}")
print(f"Slices containing multiple nodules     : {total_multi_nodule_slic
print(f"Percentage of multi-nodule slices      : {multi_nodule_percentage
print(f"Scans containing multi-nodule slices   : {multi_nodule_scan_count
print(f"Maximum nodules in a single slice      : {max_nodules_per_slice:,

display(multi_nodule_slices)
```

```
Slices containing at least one nodule : 5,472
Slices containing multiple nodules     : 183
Percentage of multi-nodule slices      : 3.34%
Scans containing multi-nodule slices   : 43
Maximum nodules in a single slice      : 3
```

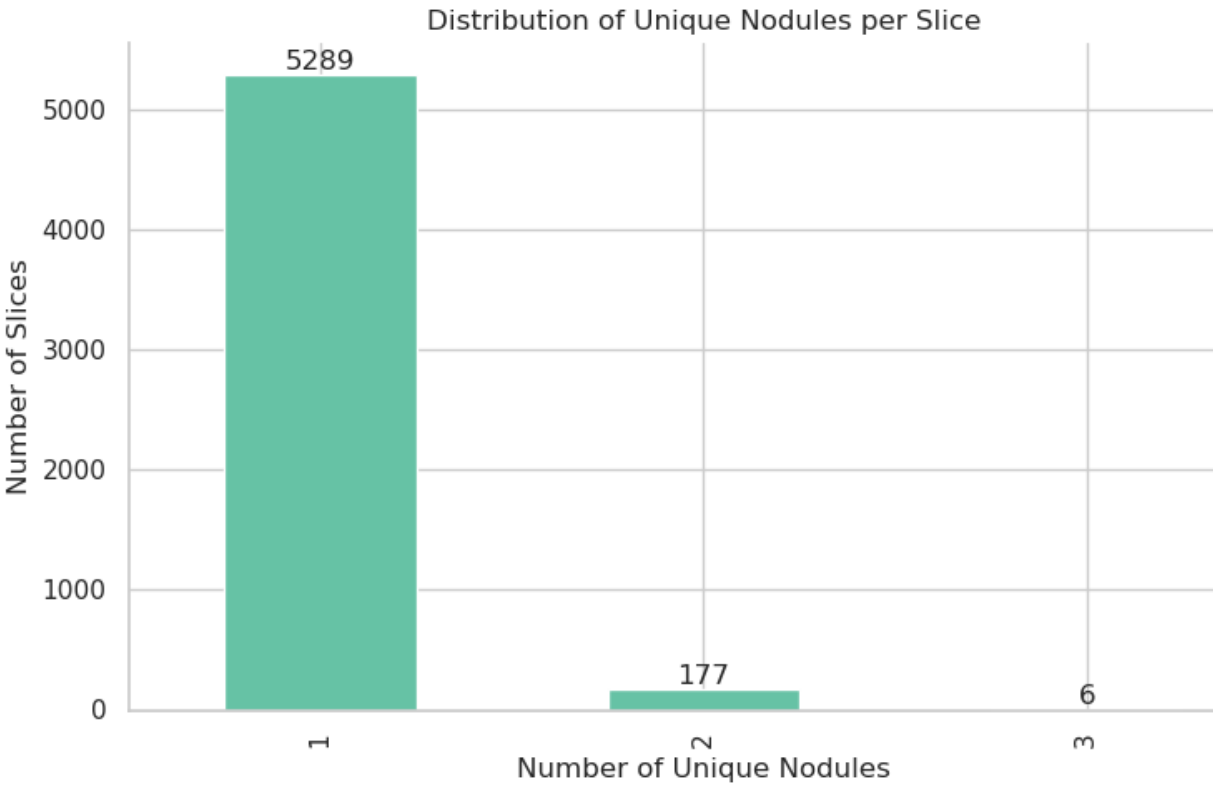
	patient_id	study_instance_uid	s
0	LIDC-IDRI-0049	1.3.6.1.4.1.14519.5.2.1.6279.6001.289514702605...	1.3.6.1.4.1.14519.5.2.1.6279.60
1	LIDC-IDRI-0049	1.3.6.1.4.1.14519.5.2.1.6279.6001.289514702605...	1.3.6.1.4.1.14519.5.2.1.6279.60
2	LIDC-IDRI-0049	1.3.6.1.4.1.14519.5.2.1.6279.6001.289514702605...	1.3.6.1.4.1.14519.5.2.1.6279.60
3	LIDC-IDRI-0049	1.3.6.1.4.1.14519.5.2.1.6279.6001.289514702605...	1.3.6.1.4.1.14519.5.2.1.6279.60
4	LIDC-IDRI-0649	1.3.6.1.4.1.14519.5.2.1.6279.6001.243200997268...	1.3.6.1.4.1.14519.5.2.1.6279.60
...	...	...	
178	LIDC-IDRI-1002	1.3.6.1.4.1.14519.5.2.1.6279.6001.233360976397...	1.3.6.1.4.1.14519.5.2.1.6279.60
179	LIDC-IDRI-1003	1.3.6.1.4.1.14519.5.2.1.6279.6001.314119990938...	1.3.6.1.4.1.14519.5.2.1.6279.60
180	LIDC-IDRI-1003	1.3.6.1.4.1.14519.5.2.1.6279.6001.314119990938...	1.3.6.1.4.1.14519.5.2.1.6279.60
181	LIDC-IDRI-1003	1.3.6.1.4.1.14519.5.2.1.6279.6001.314119990938...	1.3.6.1.4.1.14519.5.2.1.6279.60
182	LIDC-IDRI-1003	1.3.6.1.4.1.14519.5.2.1.6279.6001.314119990938...	1.3.6.1.4.1.14519.5.2.1.6279.60

183 rows × 6 columns

In [15]:
```
nodule_count_distribution = (
    slice_nodule_counts["nodule_count"]
    .value_counts()
    .sort_index()
)

ax = nodule_count_distribution.plot(
    kind="bar",
    figsize=(8, 5),
```

```
    color=sns.color_palette("Set2")[0],
)
ax.set_title("Distribution of Unique Nodules per Slice")
ax.set_xlabel("Number of Unique Nodules")
ax.set_ylabel("Number of Slices")
ax.bar_label(ax.containers[0])
sns.despine()
plt.tight_layout()
plt.show()
```



Findings

Yes. In the current cleaned metadata, **183 of 5,472 slices (3.34%)** containing consensus nodule foreground contain more than one distinct nodule. These slices occur in **43 scans**, and the maxi **three nodules in one slice**. The distribution is 5,289 slices with one nodule, 177 slices with two and 6 slices with three nodules.